

Java HotSpot VM Cheatsheet

L0 Essence: HotSpot VM 的本质是通过基于汇编模板（Template）的高速字节码译码、C1/C2 分层 JIT 编译以及 speculative 优化/去优化机制执行代码，利用 JMM 逻辑屏障在弱硬件多核一致性上提供有序性与可见性保证，并配合 Loom 协程 Mount/Unmount 用户态切换以及 ZGC 读屏障指针自愈实现极低停顿的内存周转。

[M1.1] HotSpot 类加载阶段与双亲委派机制底层实现

类加载过程包括加载（获取二进制流）、链接（验证字节码格式、为静态变量分配空间并赋予默认值、解析符号引用为直接引用）及初始化（执行 `<clinit>` 静态构造器）。双亲委派在 `ClassLoader.loadClass()` 中实现：首先查询已加载缓存，若无，则递归委派 `parent` 加载器，仅当父加载器宣告失败（抛出 `ClassNotFoundException`）时，才调用本加载器的 `findClass()`，这从底层保护了 `java.lang.*` 核心类库的安全和单例。

[M1.2] 运行时元空间（Metaspace）与类元数据（Klass）布局

从 JDK 8 开始，永久代（PermGen）被废除，取而代之的是使用本地堆内存（Native Memory）的元空间（Metaspace）。每个 Java 类在 JVM 中都对应一个 C++ 结构体 `Klass`，用于描述类的虚函数表、字段布局和继承关系。Java 对象的堆内存头部包含 Mark Word 与 Klass Word，其中 Klass Word 直接存储指向 Metaspace 中对应 `Klass` 物理地址的指针，从而实现运行期对象类型的快速查询。

[M1.3] 字节码解释器实现：模板解释器（Template Interpreter）加速原理

常规的字节码解释器采用巨大的 `switch(bytecode)` 循环，在译码和分发时会产生严重的 CPU 分支预测失败及函数调用开销。HotSpot 采用模板解释器（Template Interpreter）：在 JVM 启动阶段，动态生成器遍历所有字节码，将其直接翻译成一小段高效的底层汇编器指令模板（Template），通过专用 CPU 寄存器传递操作数（如 x86 下的 `r13` 充当字节码指针），利用尾部的直接跳转（Direct Threaded Code）实现极致的执行效率。

[M1.4] 栈帧（Stack Frame）物理布局与局部变量表操作数栈交换

HotSpot 在执行 Java 方法时在物理栈上构建栈帧（Frame）。栈帧头部保存有动态链接、方法出口和当前方法的 `Klass` 指针。局部变量表（Local Variables）是以数组形式存储方法参数与局部变量的槽位；操作数栈（Operand Stack）作为临时计算中转站。执行计算时，数据从局部变量表装载（如 `iload`）推入操作数栈项，计算完毕后（如 `iadd`）弹出计算结果，再存回（如 `istore`）局部变量表，实现频繁的栈与寄存器交换。

[M2.1] JMM 认识论：有序性、可见性、原子性与 Happens-Before 原则

由于现代多核 CPU 各自持有写缓冲区（Store Buffer）与高速缓存，硬件层面表现为弱一致性模型。Java 内存模型（JMM）是基于硬件之上的抽象软件规范，以解决多线程下的可见性、有序性和原子性问题。Happens-Before 是 JMM 的核心认识论：它定义了一组偏序关系（如 Volatile 写 Happens-Before 于后续读，锁定 Happens-Before 解锁放），如果在两个操作之间成立，则 JVM 必须向开发者保证其内存可见性与执行保序。

[M2.2] 指令重排（Instruction Reordering）与 CPU 乱序执行的物理成因

为了最大化利用 CPU 执行流水线并减少停顿，JIT 编译器会在不影响单线程语义的原则下对字节码进行重排（如优化局部变量的读写顺序）；而在多核硬件层面，CPU 采用 Out-of-Order（乱序执行）技术提前执行就绪的指令，并利用 Store Buffer 异步缓冲写入以提高吞吐。然而，这种异步写会导致数据在多核 Cache 间传播延迟，引发严重的并发数据读写顺序错乱与可见性破坏。

[M2.3] 硬件级内存屏障（Memory Barriers / Fences）设计

为提供强同步手段，多核 CPU 在硬件指令集中引入了内存屏障指令。在 x86 架构下，屏障包括 `lfence`（阻止读读/读写重排）、`sfence`（阻止写写重排）与 `mfence`（全能屏障，开销最大，清空 Store Buffer 并使其他 CPU 缓存行失效）。在 ARM 架构中，则提供 `dmb` 和 `dsb` 屏障指令。它们物理上切断了屏障前后的指令越界重排，强制使本地 Store Buffer 刷新以确保全局有序。

[M2.4] JVM 四类屏障（LoadLoad, LoadStore, StoreStore, StoreLoad）与 Volatile 实现

JVM 规范将硬件屏障抽象为四类逻辑屏障：
• `StoreStore`：确保之前的写在后续写之前对全局可见。
• `LoadLoad`：确保之前的读在后续读之前完成。
• `LoadStore`：确保读在写之前完成。
• `StoreLoad`：强制之前的写对后续读可见。
在 volatile 写之前插入 `StoreStore`，写之后插入 `StoreLoad`（在 x86 平台常被编译成 `lock addl ...` 以强行清空本地 Store Buffer）；volatile 读之后则插入 `LoadLoad` 与 `LoadStore`，彻底阻止了重排并保证多核可见。

[M3.1] 分层编译（Tiered Compilation）与 C1/C2 编译器协作

HotSpot 引擎为了平衡系统启动速度与长期运行的最佳性能，采用分层编译模型：
• `Tier 0`：解释执行阶段（Template Interpreter）。
• `Tier 1-3`：C1 编译器（Client Compiler）阶段，进行快速 JIT 编译并收集运行时 Profiling 信息（如分支跳转概率、类结构反馈）。
• `Tier 4`：C2 编译器（Server Compiler）阶段，基于 C1 收集的 Profile 执行极其激进的全局控制流/数据流编译优化，生成极高吞吐的原生机器码。

[M3.2] 编译阈值与 JIT 热度触发计算公式

JVM 通过为每个方法分配两个核心计数器判定其是否为“热点”：方法调用计数器（Invocation Counter）和回边计数器（Backedge Counter，用于监控循环次数）。在分层编译下，触发编译的公式动态结合 JIT 编译队列的负载：满足 `InvocationCounter * 4 * BackedgeCounter > Threshold * S`（其中 `S` 为与队列长度挂钩的缩放系数）时，主线程便向编译 Broker 投选异步编译请求，晋升至下一层编译。

[M3.3] 逃逸分析（Escape Analysis）与标量替换（Scalar Replacement）

在 C2 编译 Passes 中，Escape Analysis 静态扫描方法中新创建对象的指针传播路径。若分析得出该对象绝不会逸出当前线程（即不被赋值给全局变量、未作为返回值传出），则 JIT 自动执行标量替换（Scalar Replacement）：不执行堆上的物理对象分配和垃圾回收，而是把对象的各个成员字段直接拆解为栈帧上的独立局部标量，甚至直接映射到 CPU 寄存器，从而将对象分配开销完全消除。

L1 Logical Framework

• `Java Bytecode` 模板解释与元数据解耦：类加载元数据 `Klass` 常驻于本地堆的 Metaspace，解释器利用动态汇编模板（Template）消除常规 `switch` 译码开销并高速传递栈帧数据。
• 多核可见与屏障保障：JMM 通过 Happens-before 提供弱一致性保证，volatile 读写点由 JVM 强行注入硬件内存屏障，强冲刷 Store Buffer 以保证跨核可见性与有序性。
• 分层编译与退避去优：JIT 编译器采用 C1 快速收集与 C2 深度优化的多级协作，当编译期激进优化假定被运行期打破时，执行 deoptimization 物理重构帧安全滑落至解释器。
• `read barrier` 读障自愈与协程 Mount 卸载：ZGC 借助彩色指针与读屏障拦截实现并发 Relocate 下的指针自愈，Project Loom 通过在 carrier 线程挂载卸载虚拟机栈帧实现高效的用户态协程调度。

L2 Data Flow Topology

• `Incremented Machine Code` 触发器
• `Triggers C1/C2 JIT` 推测性优化
• `Speculative Optimization` 编译
• `Store Buffer Flush` 硬件缓存一致性
• `Type Guard Fails` 类型守卫失败
• `Deoptimization Trap` 重建解释器帧
• `Bailout to Interpreter` 对象分配
• `ZGC Relocate Set` 并发复制
• `Write to Forwarding Table` 线程读取旧指针（标记）
• `Load Barrier Intercept` 查找转发表
• `Self-heal Pointer to Remapped` 返回活动引用。

Trade-off Matrix: Lock & Compilation

• 设计维度（Design Dimension）：方案 A（Approach A）→ 方案 B（Approach B）[折衷折度（Trade-off Balance）]
• 锁同步策略 vs 系统级停顿：偏向锁机制（Biased Locking）→ 直接轻量级锁 CAS 竞争机制 [偏向锁在大单线程重入同步块时完全免除 CAS 指令 □ 但在多线程并发竞争激烈环境下，偏向锁撤销必须在全局安全点（SafePoint）挂起线程，导致 STW 额外停顿。JDK 15+ 默认废弃偏向锁，以消除 SafePoint 停顿开销并优化并发响应 [牺牲极少单线程执行偏置，换取多核高并发下的延迟稳定性]]
• 读屏障机制 vs 额外执行开销：Brooks Pointer 转发指针（Shenandoah GC）→ 64 位彩色指针与硬件拦截自愈（ZGC）[Brooks Pointer 通过在每个对象头前置指针简化了重定向逻辑，对 JIT 编译友好 □ 但读写均多一次解引用，且占用对象头物理空间；ZGC 染色指针将元数据压缩在指针高位内，利用读屏障在 Load 时进行自愈，读之后的 Field Access 走 native 寄存器 FastPath [牺牲了指针的高位段，换取了对象头空间的精简与 FastPath 无解引用开销]]
• 线程执行模型 vs 上下文切换：内核线程 1:1 平台模型（Platform Threads）→ 用户态轻量级虚拟线程（Project Loom）[平台线程直接映射 OS 内核线程，受操作系统直接调度，支持完整的 JNI 并发 native 块执行 □ 但虚拟线程通过堆帧冻结/解冻技术（Freeze/Thaw）在用户态调度，彻底避开了内核切换开销，支持百万级并发；代价是 synchronized 和 Native Code 会导致 Pinning 固定阻塞 [牺牲了 synchronized 重用兼容，换取了百万并发的吞吐量跃升]]
• JIT 优化策略 vs 启动预热：C2 优化编译器直接 JIT 编译 → 解释执行、C1/C2 分层 JIT 编译 [直接 C2 编译能以最短时间生成高吞吐机器码 □ 但解释执行与 C1 快速介入能大幅缩减应用首屏启动等待时间；通过分层编译与动态阈值计算（Tiered Compilation），使系统在预热期收集 Profile 并在运行期逐步升级优化 [牺牲了前期的编译队列 CPU 带宽，换取了快速启动与峰值吞吐的双重优化]]

[M4.1] 对象头 Mark Word 的物理布局与锁标记位状态

HotSpot 中每个 Java 对象头部的 Mark Word 长度与 CPU 位宽一致（32/64 位）。它在不同锁状态下复用存储位：无锁状态下存储 hashCode、分代年龄与锁标记位 01；偏向锁状态下复用空间存储获得偏向的物理线程 ID、偏向标记 1 与锁标记 01；轻量级锁状态下，整个 Mark Word 被覆写为指向当前线程执行栈中 BasicObjectLock 结构的指针，锁标记为 00；重量级锁状态下指向 C++ 内部的 ObjectMonitor 指针，锁标记为 10。

[M4.2] 偏向锁（Biased Locking）的获取、撤销与 JDK 废弃背景

偏向锁假设有锁资源大多被单一线程长期占有。首次获取锁时，通过一次 CAS 将本线程 ID 写入 Mark Word，后续该线程重入时无须任何同步指令（耗时几乎为零）。但若出现多线程竞争，撤销偏向锁必须等待全局安全点（SafePoint）暂停持有锁的线程，然后将其升级为轻量级锁。由于在高并发、多微服务的现代环境下，SafePoint 撤销带来的 STW（停顿）开销往往大于其节省的同步开销，因此偏向锁在 JDK 15 之后被默认禁用和废弃。

[M4.3] 轻量级锁（Lightweight Locking）与 CAS 栈锁记录（BasicObjectLock）

当偏向锁不可用或遭遇竞争时，锁升级为轻量级锁。加锁线程在当前的调用栈帧中分配一段 BasicObjectLock 锁记录（Lock Record）。JVM 利用 CAS 指令尝试将对象头的 Mark Word 覆写为指向该 Lock Record 的指针。若 CAS 成功，则代表当前线程无冲突地获得了轻量级锁；若 CAS 失败，代表存在多线程竞争，锁准备膨胀进入重量级锁阶段。

[M4.4] 重量级锁（Heavyweight Locking）与 ObjectMonitor 锁膨胀模型

轻量锁竞争失败后，锁会升级为重量级锁。JVM 调用 ObjectSynchronizer::inflate() 将锁膨胀，分配一个 C++ 的 ObjectMonitor 结构体，并将对象头的 Mark Word 指向该 Monitor。Monitor 内部维护 EntryList 链表挂载未抢到锁的阻塞线程，以及 WaitSet 链表挂载因 wait() 挂起的线程，最终通过操作系统的 Mutex 互斥量实现线程的物理挂起与内核级唤醒。

[M5.1] ZGC 64 位虚拟地址空间彩色指针（Colored Pointers）布局

ZGC 不使用传统对象头存储 GC 元数据，而是直接将信息存储在 64 位指针的高位中。彩色指针布局如下：低 42 位作为对象的实际物理堆寻址（最大支持 16TB 堆空间），高位中划出 4 位作为颜色标记：Marked0、Marked1（标志对象是否可达，两代交替防老标记混淆）、Remapped（表明指针已更新为对象迁移后的新物理地址）与 Finalizable。

[M5.2] ZGC 并发转移（Concurrent Relocate）与转发表（Forwarding Table）

在 Relocate 阶段，ZGC 将待回收页（Relocation Set）中的存活对象拷贝到全新的内存页中，消除碎片。由于该过程是与 Java 应用线程并发运行的，为了在对象搬迁时保证线程能找到新对象，ZGC 在旧页的 Forwarding Table（转发表）中原子记录旧对象物理地址到新对象物理地址的映射，依靠 CAS 解决 GC 线程与应用线程的拷贝竞态。

[M5.3] 读屏障（Load Barrier）的硬件指令级拦截与自愈（Self-healing）

当 Java 线程试图从字段读取一个对象引用时，读屏障会被触发：\textbf{Fast Path}：检查指针颜色位。若发现已经是 Remapped，说明安全，直接解引用访问（仅占几条汇编指令）。\textbf{Slow Path}：若颜色位是 Marked 且与当前 GC 期望不符，说明引用未被重定向。读屏障拦截执行，检索该页的 Forwarding Table，获得新对象地址并更新当前引用的指针（称为 \textbf{Self-healing} 指针自愈机制），然后将指针颜色标记为 Remapped，后续读便是 Fast Path。

[M5.4] ZGC 内存页（Page）划分与页面粗化（Fine-grained Pages）管理

ZGC 放弃了传统的新生代/老生代固定物理边界，将堆空间划分为动态调整的三种 Page：\textbf{Small Page} (2MB)：用于分配大小不超过 256KB 的小对象。 \textbf{Medium Page} (32MB)：用于分配小于等于 4MB 且大于 256KB 的中型对象。 \textbf{Large Page} (大小可变，必须为 2MB 整数倍)：每个 Large Page 仅包含一个大于 4MB 的超大对象。Large Page 不参与 Relocate 的并发转移，仅做标记和原位回收，以避免拷贝巨型对象的昂贵带宽开销。

[M6.1] G1 垃圾回收器 Region 划分与 Remembered Set (RSet) 卡表

G1 回收器打破传统的连续代区限制，将堆划分为数千个等大的独立 Region。为了能够单独回收部分 Region（Mixed GC）而不用扫描全堆，每个 Region 维护一个 Remembered Set (RSet)，用以记录“有哪些外部 Region 引用了本 Region 内的对象”。在并发写操作时，写屏障会拦截该指针修改并记录到 RSet 中，GC 标记时仅需将 RSet 加载为 Root 集合。

[M6.2] Shenandoah GC 垃圾回收器：Brooks Pointers 转发指针与 CAS 竞争

Shenandoah 回收器通过 Brooks Pointer 转发指针实现并发转移。它在对象的真实数据前增加了一个额外的指针槽。正常情况下，Brooks Pointer 指向对象自身；在 GC 并发 Relocate 迁移对象时，GC 首先拷贝一份新对象，并通过 CAS 原子操作尝试将旧对象的 Brooks Pointer 指向新对象。一旦成功，所有尝试读写旧对象的应用线程都会通过 Brooks Pointer 自动被硬件级重定向至新对象。

[M6.3] 虚拟线程（Virtual Threads / Project Loom）的设计宗旨与 carrier 线程

Java 传统的平台线程（Platform Threads）与操作系统内核线程呈 1:1 物理映射，这导致线程的创建、销毁与上下文切换涉及昂贵的内核态切换。Project Loom 引入虚拟线程（Virtual Threads），其为用户态轻量级线程，数百万条虚拟线程通过底层的 ForkJoinPool 任务队列复用极少数平台线程（此时称为载体线程 Carrier Threads），大幅提升了高并发 I/O 密集型应用的吞吐极限。

[M6.4] 虚拟线程的挂起与恢复：Mount/Unmount 状态机迁移

当虚拟线程执行阻塞的 JVM 托管 I/O（如 Socket.read()）或调用 LockSupport.park() 时，JVM 不会阻塞 Carrier 线程。JVM 拦截该调用，执行 \textbf{Unmount}（卸载）操作：将该虚拟线程的调用栈帧（Stack Frames）从平台线程的物理栈拷贝至堆内存中挂起。当底层 I/O 收到就绪事件时，JVM 调度器执行 \textbf{Mount}（挂载）操作：将栈帧从堆重新拷贝到一个空闲 Carrier 平台的栈中，无缝恢复执行。

[M6.5] 虚拟线程固定（Pinning）的物理成因与规避策略

若虚拟线程在 synchronized 块/方法内执行阻塞操作，或是在调用本地方法（JNI / Native Code）时，该虚拟线程会被强行“固定”（Pin）在当前的 Carrier 线程上，无法执行 Unmount 操作。这会导致底层的 Carrier 平台线程和 OS 线程被同步物理阻塞，若 Carrier 线程耗尽则会引发系统级饥饿。规避策略是改用 JUC 的 ReentrantLock 锁。

[M6.6] 垃圾回收的并发标记屏障（SATB）与三色标记同步

为了在 GC 线程并发标记时防止用户线程修改指针导致漏标话对象，G1 采用 SATB（Snapshot-At-The-Beginning，起始快照）技术。当写屏障拦截到对象引用的覆盖删除时，强制将即将被删除的旧引用集为灰色并记录到 SATB 的本地缓冲区队列中。GC 在标记时认为这些旧引用在快照开始时皆为活对象，从而规避了漏标，确保了并发标记的一致性。

[M6.7] 垃圾回收卡表（Card Table）的脏卡标记（Dirty Cards）过程

HotSpot 的老年代内存按 512 字节划分为无数个逻辑 Card。对应的 Card Table 字节数组中，每个字节映射一个 Card 的存活状态。当应用线程执行写操作（如 obj.field = value）跨越指向新生代时，写屏障执行 card_table[addr >> 9] = 0，将该 Card 标记为 Dirty。新生代 Minor GC 时直接扫描 Dirty 卡中的指针，免去遍历整个老年代。

[M6.8] HotSpot 安全点（SafePoint）与安全区域（Safe Region）轮询机制

为了执行 GC 或锁撤销等全局操作，JVM 必须在 SafePoint 挂起所有线程。Java 线程在循环末尾、方法返回和调用出口处预埋轮询指令。通过主动读取一个特定的全局 SafePoint 轮询页（在需要 STW 时，JVM 将该页设为只读以触发硬件保护段异常，线程捕获异常后将自身挂起）。对于 sleep 或 blocked 的线程，由于不执行字节码，它们被认为处于 Safe Region，GC 可以在不阻拦它们的情况下并发回收。

Zone T1: HotSpot parameters & flags

-XX:+UnlockDiagnosticVMOptions -XX:+PrintAssembly：打印 JIT 编译器（CI/C2）

编译出来的机器码汇编指令流（需配合 hsdis 动态链接库）。\XX:+UseZGC / -XX:+UseGenerationalZGC：开启低延迟 ZGC 垃圾回收器，或者开启性能更强的前后台分代 ZGC。 \XX:+TraceDeoptimization：跟踪并打印 JIT 去优化（Deopt）的详细原因、方法签名和栈帧重构日志。 \XX:CompileThreshold = 10000：设置方法调用计数器阈值，当方法和循环回边累计达到该值时，JVM 启动 C2 编译。 \XX:-UseBiasedLocking：显式禁用偏向锁优化，直接使用轻量级锁以平滑锁升级停顿。

Zone T2: Fences & Lock diagnostics

lock addl \$0x0, (%rsp) / lock orl \$0x0, (%rsp)：x86 架构下 volatile 写操作后发射的指令。通过 lock 前缀指令强制清空 CPU Store Buffer 并使其他核的 L1/L2 缓存行失效，起到 StoreLoad 逻辑屏障的作用。 \mov 0x8(%rdi), %r10 \cmp %r10, MAP_SMI \jne deopt_stub：JIT 编译的对象 Map 检查特征。先提取 Class Pointer，再与预配的 Klass 地址比较，失败则跳转到去优化 Stub。 \0x0000000000000005 (MarkWord 末尾 = 105)：代表当前对象处于重量级锁状态，MarkWord 存储着 ObjectMonitor 的物理地址。 \test %eax, 0x160000(%rip) (SafePoint Poll)：线程退出或回边时执行的安全点轮询指令。JVM 将该地址映射为只读时，执行此读取会触发 SIGSEGV 信号，从而让线程在安全点自动挂起。